DEPLOY STATIC WEBSITE PROJECT

Monday, March 23, 2026 3:48 PM

Deploy secure, low-latency static website with DNSSEC for Amazon Route 53 + CloudFront + WAF + Encryption (SSL/TLS) using Terraform

Project overview

This Terraform project demonstrates how to deploy a secure static website on AWS using Terraform with Amazon S3, CloudFront, Route 53, AWS Certificate Manager (ACM), AWS KMS and AWS WAF.

AWS offer flexible pricing options of Content Delivery Network (CDN) with CloudFront, provide easy deployment options to start. If you are accessing any enterprise level website or watching videos, you are accessing content from CDN. CDN can not only help by adding caching, but provide options to increase security, improve high availability, save bandwidth your origin load on your servers. This is where Amazon CloudFront becomes your secret weapon. It's not just another AWS service; it's the difference between a sluggish website and a lightning-fast user experience that keeps customers coming back.

The challenge

Hosting static websites on traditional servers requires managing OS patches, web server configuration, SSL renewal, and scaling—tasks that add no value for static content. **Amazon S3**, **CloudFront**, and **Route 53** remove this overhead while providing global CDN distribution, managed SSL, and DNS.

Solution overview

This Terraform project demonstrates production-ready AWS CloudFront distribution using modern AWS practices. The solution implements a three-steps deployment architecture that separates DNS management, infrastructure provisioning, and content deployment for better maintainability and security.

In this project, I used Amazon S3 to **store** website files, Amazon CloudFront to **cache** and **serve** them globally over HTTPS, and Amazon Route 53 to **route** the custom domain to CloudFront. Together, they form a complete static hosting platform. The setup also includes **ACM** for SSL certificates, **AWS KMS** encryption for storage, AWS WAF to protect against attacks and **DNSSEC** for DNS security.

Architecture overview

This solution consists of three dependent steps:

- **Step 1** provisions the Route 53 hosted zone and enables DNSSEC.
- **Step 2** deploys the S3 buckets, CloudFront distribution, SSL certificate and enable WAF to protect against attacks – using the hosted zone from **Step 1** for DNS validation.

- **Step 3** handles content deployment separately, updating website content without re-provisioning infrastructure, and relies on the infrastructure built in Step 2.

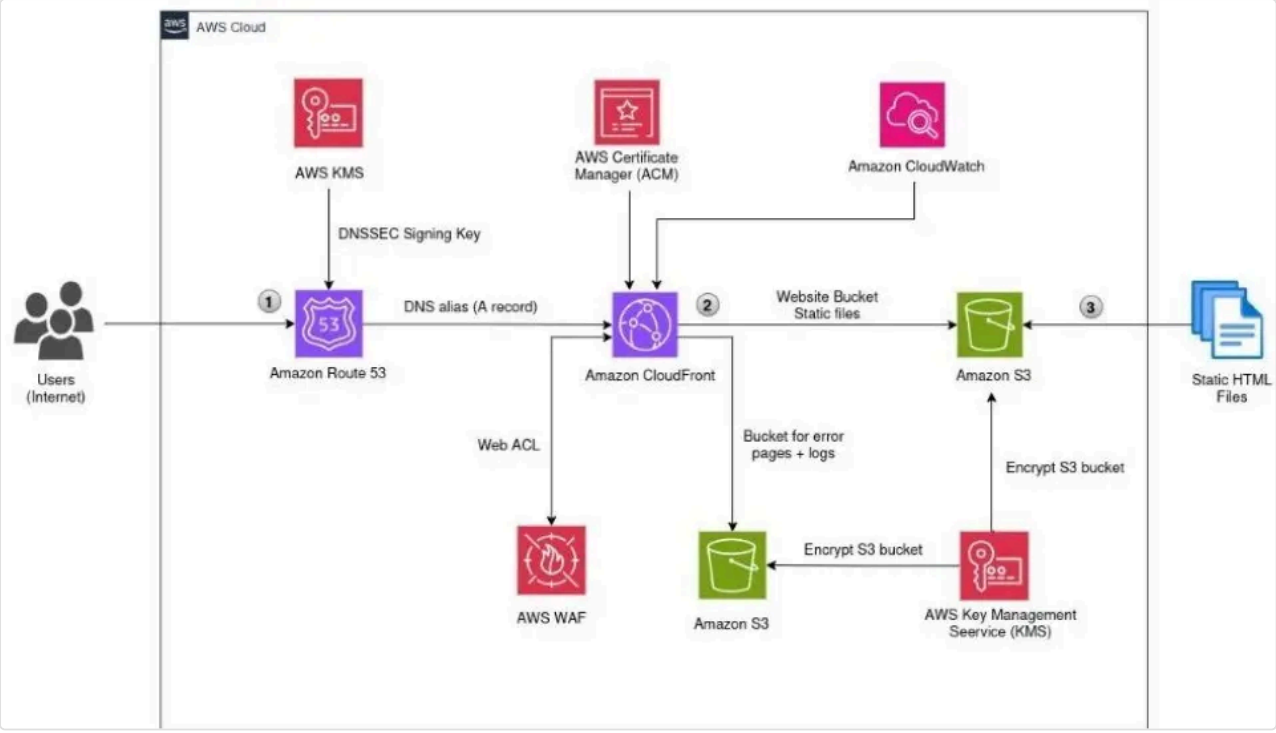


Figure 1 — Three-stage deployment architecture: DNS, Infrastructure, and Content layers

Tech Stack

The solution uses the following AWS services:

- **Amazon Route 53** — Hosted zone with A records (root + www) and DNSSEC enabled
- **Amazon CloudFront** — CDN distribution with Origin Access Control (OAC), HTTPS redirect, custom error pages, and access logging
- **Amazon S3** — Two private buckets: one for website content, one for error pages and CloudFront access logs
- **AWS Certificate Manager (ACM)** — SSL/TLS certificate with DNS validation
- **AWS KMS** — Encryption keys for S3 buckets, SSM parameters, and DNSSEC signing
- **AWS WAF** — Enable WAF to protect against attacks.

Traffic Flow

This diagram shows a secure, scalable static website hosting setup on AWS. Here's how it flows:

1. User → Route53 → CloudFront (HTTPS / TLS 1.2/1.3)
2. CloudFront → WAF → S3 (HTTPS/TLS 1.2/1.3)
3. Response → S3 → CloudFront → User

Request Flow (Steps 1 → 2 → 3)

- **Step 1 — DNS Resolution (Users → Route 53)** Users type a URL in their browser. Amazon Route 53 handles DNS resolution via an **A record (DNS alias)**. Route 53 also uses **AWS KMS** to manage a **DNSSEC signing key**, protecting against DNS spoofing attacks.
- **Step 2 — Content Delivery (Route 53 → CloudFront)** Traffic is routed to **Amazon CloudFront** (the CDN), which:
  - Uses an **SSL/TLS certificate** from AWS Certificate Manager (ACM) for HTTPS
  - Is monitored by **Amazon CloudWatch** for metrics and alarms
  - Is protected by **AWS WAF** (Web Application Firewall) via a **Web ACL**, filtering malicious traffic
  - Sends error pages and access logs to a dedicated **Amazon S3 bucket** (also KMS-encrypted)
- **Step 3 — Content Storage (CloudFront → S3)** CloudFront fetches static files from a **primary Amazon S3 bucket** (the Website Bucket). The static HTML files are uploaded here by developers/CI pipelines. This bucket is also **encrypted using AWS KMS**.

Key Security Layers

Layer	Service	Purpose
DNS Security	KMS + Route 53	DNSSEC protection
Transport	ACM	HTTPS/TLS certificates
Edge Protection	AWS WAF	Block malicious requests
Storage Encryption	AWS KMS	Encrypt both S3 buckets
Observability	CloudWatch	Monitor & alert

- **SSL/TLS for Data in Transit**
  - HTTPS (TLS 1.2/1.3) between User <---> CloudFront
  - TLS 1.2/1.3 between CloudFront <---> S3
- **Amazon CloudFront (CDN)**
  - Global Edge Locations
  - Caching (Images, CSS, JS, Articles)
  - DDoS Protection (Edge)
  - HTTPS (TLS 1.2/1.3)
- **AWS WAF (Web Application Firewall)**
  - OWASP Top 10 Protection
  - SQL Injection Protection
  - XSS Protection
  - IP Rate Limiting
  - Geo Blocking
  - Bot Control
- **Amazon S3 (Encrypted)**

    - SSE-S3 / SSE-KMS (AES-256)
    - Encryption Keys managed by AWS KMS
- **AWS Certificate Manager (ACM)**
    - SSL/TLS Certificates
    - SSL/TLS Certificate from AWS Certificate Manager (ACM)
    - Used by CloudFront
    - Enforces HTTPS (443)
    - Supports TLS 1.2 and TLS 1.3
- **Security Highlights**
    - Only HTTPS (443) allowed
    - WAF protects from common attacks
    - All data in transit encrypted (TLS)
    - All data at rest encrypted
    - Least privilege access
    - Centralized key management (KMS)
- **Monitoring & Supporting Services**
    - Amazon CloudWatch (Monitoring & Logs)
    - AWS CloudTrail (Audit Logs)
    - Key Management AWS KMS (Keys Encrypted & Rotated)

This is a well-architected, **serverless** static hosting pattern — no EC2 instances needed, highly available by default, and secure end-to-end.

Key features

Three-Stage Deployment Architecture

- Isolated DNS, infrastructure, and content deployment workflows
- Deploys the S3 buckets, CloudFront distribution, SSL certificate and WAF
- Independent state management for each deployment layer

Enterprise-Grade Security

- Private S3 buckets with Origin Access Control (OAC) and SigV4 signing
- End-to-end encryption: KMS keys for S3, SSM parameters, and DNSSEC signing
- TLS 1.2 minimum with automatic HTTPS redirect
- DNSSEC-enabled Route 53 hosted zone for DNS integrity
- OIDC authentication for GitHub Actions (no long-lived credentials)
- AWS Web Application firewall (WAF) to protect against attacks

Production-Ready Features

- Global content delivery via CloudFront CDN

- Custom error pages (404/500) with dedicated S3 bucket
- CloudFront and S3 access logging for audit trails
- Automated cache invalidation on content updates
- S3 versioning with 90-day retention for rollback capability
- AWS WAF to filter the malicious requests and provide protection from intruder
- Monitoring via AWS CloudWatch service to create dashboards and alarms in case of errors and performance issues

Cost & Compliance:

- **Infracost** integration for infrastructure cost estimation
- **Checkov** security scanning in CI/CD pipeline
- Automated deployment reduces manual errors and operational overhead
- To improve performance, enable compression and optimize cache key by including only required headers, cookies and query strings.
- For cost optimization, select the correct price class. Allow only geographic regions in which you want to allow users to access the application. Use different TTL for static and dynamic content. Use primary and secondary origins to increase availability. Use cache invalidation carefully.

Modular Architecture

- Reusable Terraform module
- Clean separation of concerns
- Easy to maintain and extend
- Production-ready structure

Prerequisites

- An AWS account with an IAM role configured for GitHub Actions via OIDC. This eliminates long-lived credentials.
- An Amazon S3 bucket for Terraform remote state
- Register domain through Route 53: This is a pre-requisite before we terraform code. You can buy domain from external DNS provider or buy from Route53 DNS request option.
- You have to use your name domain name part of route53 zone.

Implementation

Prerequisites

- AWS CLI — v2+ configured with `aws configure` and with appropriate permissions
- Terraform >= 1.14.8 installed
- Domain registered through Route 53 (creates hosted zone automatically)

Required AWS Permissions

Your AWS user/role needs permissions for:

- A registered domain with access to change its nameservers at your domain registrar
- Ensure the IAM role has permissions to create and manage all resources including Route 53, S3, CloudFront, ACM, KMS, and WAF
- CloudWatch (dashboard and metrics access)
- For this code to function without errors, create an OpenID Connect identity provider in Amazon Identity and Access Management that has a trust relationship with this GitHub repository.
- Store the ARN of the IAM Role as a GitHub secret named `AWS_ROLE_ARN` which is referenced in the workflow files.
- For Infracost integration in this repository, the `INFRACOST_API_KEY` needs to be stored as a repository secret.
- An S3 bucket for Terraform remote state — update the bucket value in `bootstrap/` module and `backend.tf`

Project Structure

s3-static-website/

Path	Description
— Documentation	# Documentation of this project and AWS services study note
— error-pages	# HTML file for custom error page
— modules/acm/	# This module sets up an ACM for certificate
— modules/bootstrap/	# This module sets up a backend for Terraform state file
— modules/cloudfront/	# This module creates a CloudFront for CDN
— modules/cloudwatch/	# This module creates a monitoring for CloudFront
— modules/iam/	# This module creates an IAM for this project
— modules/kms/	# This module sets up a KMS encryption for S3 bucket
— modules/route53/	# This module sets up an Amazon Route 53 for DNS and DNSSEC signing
— modules/s3/	# This module sets up a private S3 for HTML files and custom error pages + log
— modules/sns/	# This module sets up an SNS notification system for alert
— modules/waf/	# This module sets up a AWS WAF
— .github/workflows/	# GitHub Actions workflows
— website-deployment.yml	# Static Website deployment workflows
— terraform-checkov.yml	# Checkov workflows
— terraform-infracost.yml	# Infracost workflows
— backend.tf	# Backend configuration for Terraform state file
— main.tf	# Main infrastructure resources
— variables.tf	# Input variables and defaults
— dev-terraform.tfvars	# The terraform.tfvars file for dev

Required GitHub Secrets

Go to **Settings** → **Secrets and variables** → **Actions** in your fork:

Secret	Description
AWS_REGION	us-east-1
AWS_ROLE_ARN	Your github-actions-role ARN
TF_BACKEND_S3_BUCKET	Your state bucket name
TF_VERSION	1.14.8

> **Step 1: Remote state backend infrastructure and GitHub Actions CI/CD authentication using OIDC**

Module "bootstrap"

This code sets up the remote state backend infrastructure for Terraform itself — it's the "bootstrap" you run once before using remote state.

The Problem It Solves

By default, Terraform stores its state file locally. In a team or CI/CD environment you need that state stored remotely and protected. This code creates that remote storage on AWS.

Resource by Resource

1. **aws_s3_bucket.terraform_state** — Creates the S3 bucket that will store your `.tfstate` files.
2. **aws_s3_bucket_versioning.terraform_state** — Enables versioning on the bucket.
   - Every time Terraform writes a new state, the old one is preserved as a previous version
   - This lets you roll back to any prior state if something goes wrong
3. **aws_s3_bucket_server_side_encryption_configuration.terraform_state** — Encrypts everything in the bucket at rest using AWS KMS.
   - State files often contain sensitive values (passwords, keys, IPs), so encryption is critical
   - `aws:kms` uses AWS-managed keys by default (you can swap in a customer-managed key)
4. **aws_s3_bucket_public_access_block.terraform_state** — Blocks all public access to the bucket — all four settings set to true.
   - Prevents accidental public exposure of your state file
   - Acts as a hard override even if someone tries to add a public bucket policy later
5. **aws_dynamodb_table.terraform_locks** — Creates a DynamoDB table for state locking.
   - When Terraform runs, it writes a LockID entry to this table
   - If two people (or two CI jobs) run `terraform apply` simultaneously, the second one sees the lock and waits or errors — preventing state corruption
   - `PAY_PER_REQUEST` means no capacity planning needed; you only pay per lock/unlock operation

Module "iam" — ga-oidc.tf

This code sets up GitHub Actions CI/CD authentication to AWS using OIDC (OpenID Connect) — a keyless, token-based auth method. No long-lived AWS credentials stored in GitHub secrets.

Overall Flow

```
GitHub Actions job starts
    |
    ▼
GitHub issues a signed OIDC JWT token
    |
    ▼
Workflow calls sts:AssumeRoleWithWebIdentity with that token
    |
    ▼
AWS verifies the token against the registered OIDC provider
    |
    ▼
AWS checks repo/org conditions match
    |
    ▼
AWS returns short-lived credentials (max 1h)
    |
    ▼
Terraform runs with those credentials
```

Step 2: Provision the Route 53 hosted zone

We provisioned the Route 53 hosted zone, which became the domain's authoritative DNS zone once I configured its nameservers at the domain registrar (SquareSpace). We also enabled DNSSEC using an AWS KMS (Key Management System) Key Signing Key to cryptographically protect DNS responses against tampering.

DNSSEC (Domain Name System Security Extensions) is a security protocol that adds cryptographic signatures to DNS records. This ensures that users are connecting to legitimate websites rather than malicious ones. It also helps prevent attacks like DNS spoofing or cache poisoning by verifying the authenticity and integrity of DNS responses.

Amazon Route 53 is a scalable and highly available domain name system (DNS) web service that helps route end-user requests to applications hosted on AWS. Integrating Amazon Route 53 with DNSSEC tightens security to ensure the authenticity of DNS responses.

DNSSEC adds digital signatures to DNS records so that when someone looks up a domain (like amazon.com), they can verify that the DNS info came from the right source and wasn't tampered with. In AWS, that is achieved by — (i) creating an asymmetric KMS Key for DNSSEC signing, (ii) enabling it for use with Amazon Route53, (iii) associating the KMS Key with the Amazon Route53 hosted zone to create a Key Signing Key, (iv) activating DNSSEC for the hosted zone, and (v) updating the DS records with the domain registrar to link the domain to the DNSSEC chain of trust.

Module "route53"

This code sets up DNS hosting and DNSSEC security for a domain on AWS.

1. Route 53 Hosted Zone (aws_route53_zone.main)

Creates a DNS zone for your domain — essentially a container that holds all DNS records for `var.domain_name`. Tags are applied using a merge of common tags plus resource-specific ones.

2. KMS Key for DNSSEC (aws_kms_key.dnssec)

Creates a cryptographic signing key used to sign DNS records. Key details:

- `ECC_NIST_P256` — an elliptic curve algorithm required by Route 53 for DNSSEC
- `SIGN_VERIFY` usage — the key can only sign and verify, not encrypt data
- 7-day deletion window — a safety buffer before the key is permanently destroyed
- Must be in `us-east-1` (noted in the comment) — a hard AWS requirement for DNSSEC keys

3. KMS Alias (aws_kms_alias.dnssec)

Creates a human-readable name for the KMS key (e.g. `alias/myapp-example-com-dnssec`) so it's easier to identify in the AWS console.

4. KMS Key Policy (aws_kms_key_policy.dnssec)

Defines who can do what with the signing key.

5. Key Signing Key (aws_route53_key_signing_key.main)

Links the KMS key to your hosted zone as a Key Signing Key (KSK) — the actual DNSSEC signing mechanism attached to the zone. Set to ACTIVE so it immediately starts being used. The `depends_on` ensures the KMS key and its policy exist before this resource is created, avoiding a race condition.

6. Enable DNSSEC (aws_route53_hosted_zone_dnssec.main)

Turns DNSSEC on for the hosted zone. Must wait for the KSK to exist first (`depends_on`). Once applied, Route 53 will cryptographically sign all DNS responses, protecting against DNS spoofing/cache poisoning attacks.

7. DNS Records (aws_route53_record.root + .www)

Two A records that point your domain to a CloudFront distribution:

- `cloudwithdarvin.com` → CloudFront (root domain)
- `www.cloudwithdarvin.com` → CloudFront (www subdomain)

Both use alias records (AWS-native, free, and auto-updating) rather than plain A records with hardcoded IPs. `evaluate_target_health = false` means Route 53 won't check if CloudFront is healthy before responding.

8. Overall Architecture

```
cloudwithdarvin.com (Route 53 Zone)
    └── DNSSEC enabled (KMS key signs all responses)
        ├── example.com     ──alias──▶ CloudFront Distribution
        └── www.example.com ──alias──▶ CloudFront Distribution
```

This is a solid, security-conscious setup for hosting a static site or CDN-fronted application with DNS-level attack protection.

Step 3: Deploy S3 buckets, CloudFront distribution, and SSL certificate

The second stage focuses on creating the remaining infrastructure components.

- **Provision SSL certificate (ACM + DNS validation)**
  - CloudFront only accepts certificates from *us-east-1*, regardless of where your other infrastructure lives.
  - CloudFront is a global service, but certificates must be in the **us-east-1** region.
  - For certificate validation, you need to add the CNAME record mentioned in ACM services to prove you own the domain:
    - **ACM generates validation records** with random values
    - **You add CNAME records** to your DNS zone
    - **ACM checks the records** to verify domain ownership
    - **Certificate becomes valid** once verification succeeds

Module "acm"

This code provisions and validates an SSL/TLS certificate for a website using AWS Certificate Manager (ACM) and Route 53. It has three resources that work together in sequence.

1. aws_acm_certificate — Request the Certificate

- Requests a certificate that covers both the apex and www subdomain
- Uses DNS validation (as opposed to email), meaning AWS will verify you own the domain by checking for specific DNS records
- `create_before_destroy` ensures a new cert is provisioned before the old one is destroyed, avoiding downtime during replacements
- The commented-out `provider = aws.us_east_1` is a reminder that CloudFront requires ACM certs to live in **us-east-1**, regardless of your default region

2. aws_route53_record — Create DNS Validation Records

- ACM gives back a set of `domain_validation_options` — one per domain/SAN — each containing a special DNS record AWS wants to see
- The `for_each` loop iterates over both domains (cloudwithdarvin.com and www.cloudwithdarvin.com) and creates a Route 53 record for each
- `allow_overwrite = true` prevents errors if the record already exists (e.g. from a previous apply)
- Short *ttl* = *60* helps AWS pick up the records quickly

3. aws_acm_certificate_validation — Wait for Validation to Complete

- This resource doesn't create anything — it's a Terraform gate that blocks until AWS confirms the certificate is validated
- It passes the FQDNs of the DNS records created above so Terraform knows what to watch
- The 10-minute timeout gives AWS enough time to propagate and verify the DNS records

- Other resources that depend on the cert (like a CloudFront distribution) should reference **this** resource's ARN, not the certificate directly, to ensure the cert is fully validated before use

4. The Overall Flow

```
Request cert (ACM)
    ↓
AWS returns validation DNS records
    ↓
Create CNAME records in Route 53
    ↓
AWS validates domain ownership
    ↓
Certificate becomes ISSUED
    ↓
CloudFront can now reference the validated ARN
```

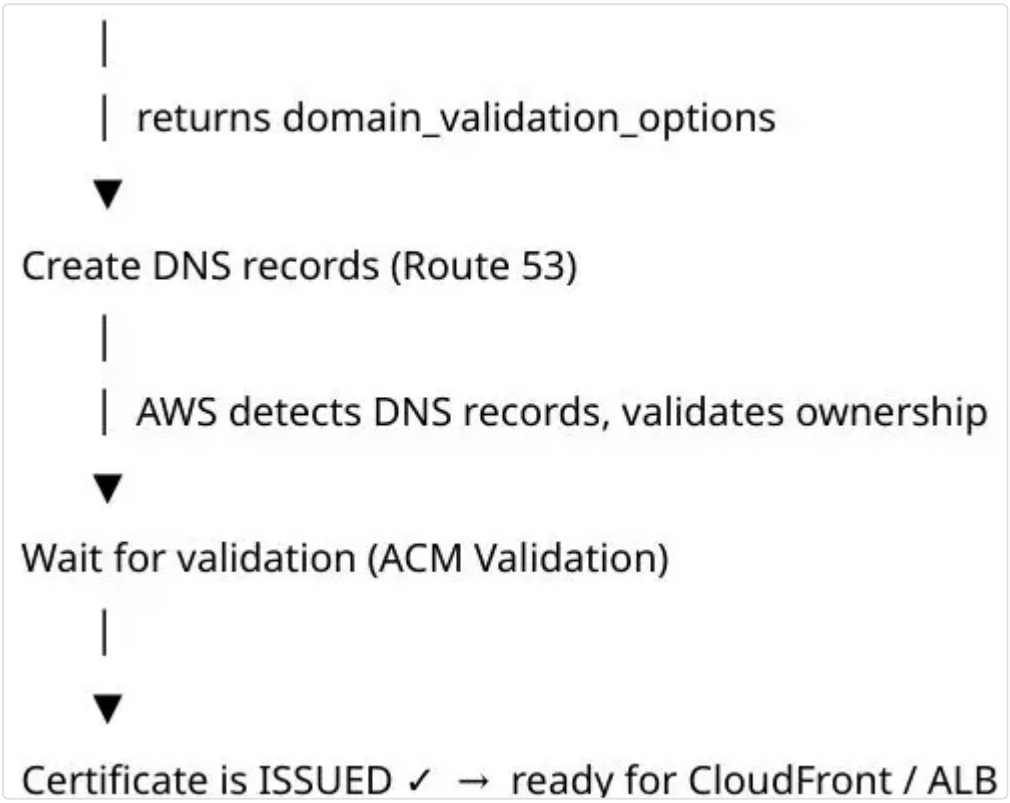


Figure 2 — ACM certificate provisioning and DNS validation flow

Create an AWS KMS key for S3 encryption:

- This setup uses two S3 buckets — one for website content and one for operational files such as error pages and access logs.
- We don't have CloudFront edge cache logs in CloudFront log groups so one option is to store them in S3 buckets. This access logs is useful in case of debugging web page and caching issues.
- CloudFront needs the `kms:Decrypt` permission to read objects from encrypted buckets via Origin Access Control.

- The key policy also grants the S3 service access to encryption/decryption and the account root full management permissions.

Module "kms"

This code sets up an AWS KMS (Key Management Service) encryption key for securing S3 bucket data.

1. aws_kms_key — Creates the KMS Key

This creates the actual encryption key with:

- A 7-day deletion window — if you delete the key, AWS waits 7 days before permanently destroying it (safety net)
- Automatic key rotation enabled — AWS rotates the cryptographic material yearly, a security best practice
- Tags merging common project tags with key-specific metadata (name, type, purpose)

2. aws_kms_alias — Creates a Human-Readable Alias

KMS keys have unwieldy IDs by default. This creates a friendly alias like `alias/myproject-encrypt-s3`, making it easier to reference the key in other services or the AWS Console.

3. aws_iam_policy_document — Defines Who Can Use the Key

This data source builds a JSON policy with three permission statements:

The root account statement is actually an AWS requirement — without it, only the key policy controls access, which could lock you out. It delegates key management back to IAM.

CloudFront gets read-only crypto permissions (just Decrypt + DescribeKey) because it only needs to serve/read encrypted objects, never write them.

4. aws_kms_key_policy — Attaches the Policy to the Key

This final resource takes the JSON policy document built above and actually attaches it to the KMS key, making the permissions active.

5. Big Picture

```
KMS Key ──► used by S3 to encrypt stored objects
        ──► used by CloudFront to decrypt objects for delivery
        ──► managed by your AWS account via IAM
```

It's a standard, well-structured setup for a CloudFront → S3 static site or media bucket where content is encrypted at rest and only authorized services can access it.

Create an S3 website bucket with encryption, versioning, and lifecycle policies

- The website bucket stores the static HTML files served through CloudFront. It uses the KMS key from previous Step for server-side encryption, blocks all public access, and enables versioning so you can roll back content changes.
- A lifecycle policy expires noncurrent versions after 90 days and cleans up incomplete multipart uploads.

- The bucket also logs S3 access requests to the ops bucket under the `access_logs/` prefix.

Module "s3" — s3-website.tf

This code provisions a secure, production-grade S3 bucket designed to host a static website (HTML, CSS, JS, images) behind CloudFront.

random_string.bucket_suffix

Generates an 8-character random lowercase alphanumeric string. Since S3 bucket names must be globally unique, appending this suffix to the bucket name avoids naming collisions across AWS accounts.

aws_s3_bucket_logging.website — Access logging

Sends access logs to a separate S3 bucket (cloudfront_ops, from a logging module). Logs are prefixed with `access_logs/`, enabling auditing of who accessed or requested objects.

aws_s3_bucket_server_side_encryption_configuration.website — Encryption at rest

Encrypts all objects using AWS KMS (`aws:kms`) with a customer-managed key (CMK). This is stronger than the default S3-managed encryption (AES-256), giving you full control over key rotation and access policies.

aws_s3_bucket_website_configuration.website — Static website hosting

Enables S3's built-in static website hosting: `index.html` is served for root/directory requests, `error.html` is served for 4xx errors. Note: even though website hosting is enabled, public access is blocked. In practice, this bucket is accessed exclusively through CloudFront, not directly via the S3 website endpoint.

aws_s3_bucket_public_access_block.website — Security hardening

Blocks all public access to the bucket at every level (ACLs and bucket policies). This is a security best practice — objects are never accidentally made public, even if a policy is misconfigured.

aws_s3_bucket_versioning.website — Object versioning

Keeps a full history of every object version. This enables rollback to a previous deployment if a bad file is uploaded, and protection against accidental deletions.

aws_s3_bucket_lifecycle_configuration.website — Cost management

Rule	Behavior
noncurrent_version_expiration	Permanently deletes old object versions after 90 days
abort_incomplete_multipart_upload	Cleans up failed/partial uploads after 7 days

Overall Architecture Summary

```
CloudFront → S3 Bucket (private, encrypted, versioned)
                ↓ logs
         S3 Logging Bucket (cloudfront_ops)
```

This follows AWS security best practices: no public access, KMS encryption, audit logging, versioning with automatic cleanup, and a randomized unique name.

Create S3 ops bucket for error pages and CloudFront access logs

- The ops bucket serves two purposes: it hosts custom error pages (404, 500) that CloudFront serves when errors occur, and it receives CloudFront access logs under the `logs/` prefix.
- Lifecycle policies transition logs to Standard-IA after 30 days, Glacier after 90 days, and delete them after one year to control storage costs.

Module "s3" — s3-logging.tf

This Terraform code provisions a single S3 bucket designed to support CloudFront operations (logs, error pages, etc.), with a full set of security and cost-management configurations.

4. aws_s3_bucket_public_access_block — Public access settings

Setting	Value	Meaning
block_public_acls	false	Allows ACLs (needed for CloudFront logging)
block_public_policy	true	Blocks any bucket policy that grants public access
ignore_public_acls	false	Honors ACLs (again, for CloudFront)
restrict_public_buckets	true	Prevents public access via policies

The intentional `false` values are the minimum relaxation needed for CloudFront log delivery, while the `true` values keep the bucket protected from accidental public exposure.

7. aws_s3_bucket_lifecycle_configuration — Cost & hygiene rules

Three rules manage objects over time:

Rule 1 — abort_incomplete_uploads (applies to everything): Automatically cancels any stalled multipart upload after 7 days, preventing orphaned data from accumulating charges.

Rule 2 — manage_log_files (prefix: `logs/`): A tiered storage strategy to reduce costs as logs age:

- Day 0–30: Standard (fast, full-price)
- Day 30–90: Standard-IA (infrequent access, ~45% cheaper)
- Day 90–365: Glacier (archival, ~80% cheaper)
- After 365 days: Deleted entirely

Rule 3 — manage_error_pages (prefix: `error-pages/`): Error pages are kept indefinitely (no expiration), but incomplete multipart uploads are still cleaned up after 7 days.

Create CloudFront Origin Access Controls

- Origin access control is a must have policy to restrict origin access by CloudFront only, preventing direct access by users or attacker. So user must access web application through CloudFront and origin will

remain private and protected. This adds another layer of security.

- Origin Access Control (OAC) is the mechanism that allows CloudFront to access private S3 buckets. Two OAC resources were created — one for the website bucket and one for the ops bucket's error pages — each using SigV4 signing to authenticate requests to origin, which is set to `s3`.

Create CloudFront distribution

- The distribution is the central resource that ties everything together. It defines two origins (the backend sources from where CloudFront fetches content): the website bucket for static content and the ops bucket for error pages.
- The distribution is configured with aliases for both the root domain and the www subdomain, and uses `index.html` as the default root object, so visitors hitting the base URL get the homepage.
- The default cache behavior routes all requests to the website bucket. It redirects HTTP visitors to HTTPS automatically.
- The **Managed-SecurityHeadersPolicy** AWS policy is also attached, which adds security headers like Strict-Transport-Security, X-Content-Type-Options, and X-Frame-Options to all responses.

Cache Policies

- Cache policies in CloudFront distribution are managed from distribution cache behavior.
- HTML Content Policy
- Static Assets Policy
- An ordered cache behavior matches the `/error-pages/*` path pattern and routes those requests to the ops bucket — a separate behavior is needed because the error pages are stored in a different origin than the website content.
- Custom error responses map the HTTP 404, 403, and 500 error codes to the corresponding error pages.
- Finally, the access logging writes to the ops bucket under the `logs/` prefix, and the ACM certificate from previous Step enables HTTPS on the custom domain.

Module "cloudfront"

This module sets up a CloudFront CDN distribution for a static website hosted on S3, with security, caching, and access control configurations.

1. Origin Access Controls (OAC)

These lock down the S3 buckets so only CloudFront can read from them — not the public directly. It uses AWS's `sigv4` signing protocol to authenticate CloudFront's requests to S3.

2. Response Header Policies

Two policies are defined:

- **security_headers** — Adds HTTP security headers to every response:
  - `X-Content-Type-Options: nosniff` — prevents MIME-type sniffing
  - `X-Frame-Options: DENY` — blocks the site from being embedded in iframes
  - `Referrer-Policy` — controls what referrer info is sent

    - **Strict-Transport-Security (HSTS)** — forces HTTPS for 1 year, including subdomains
- **cors_policy** — Allows cross-origin GET/HEAD/OPTIONS requests from any origin (wildcard *)

3. The CloudFront Distribution

The main distribution resource with:

- Aliases: `var.domain_name` and `www.var.domain_name`
- Default root object: `index.html`
- HTTP to HTTPS redirect enforced
- Minimum TLS version: TLSv1.2_2021
- WAF ACL attached via `web_acl_id`
- Geo-restriction optionally configurable via a variable

4. Cache Behaviors (in priority order)

CloudFront evaluates these top-to-bottom, using the first match:

1. `/assets/img/{bg,misc,portfolio,profile,services}/*` — dynamic block covering 5 specific image subfolders, long-term caching (up to 1 year)
2. `/assets/img/*` — catches any remaining images directly under /img/
3. `/assets/css/*.css` — stylesheets
4. `/assets/js/*.js` — JavaScript files
5. `/assets/vendor/**` — third-party vendor assets
6. `/error-pages/*` — served from the ops bucket, not the website bucket
7. Default behavior — catches everything else (HTML pages), uses a shorter cache TTL

Static assets (1–5) all use the `static_assets` cache policy with a 1-day default TTL and 1-year max TTL.

5. Cache Policies

Policy	Default TTL	Max TTL	Use case
static_assets	1 day	1 year	Images, CSS, JS
html_content	1 hour	1 day	HTML pages (defined but not actively used — it's commented out in the default behavior)

Both strip cookies, headers, and query strings from cache keys, keeping caching efficient.

6. Custom Error Pages

HTTP Error	Shown Page
404	/error-pages/404.html
403	/error-pages/404.html (treats forbidden as not found — hides internal structure)
500	/error-pages/500.html

7. S3 Bucket Policies

Two IAM policies are attached to the S3 buckets, granting CloudFront permission to:

- `s3:GetObject` from the website bucket (to serve content)
- `s3:GetObject` from `/error-pages/*` in the ops bucket
- `s3:PutObject` to `/logs/*` in the ops bucket (to write access logs)

All are scoped to this specific CloudFront distribution's ARN for least-privilege access.

Summary Architecture

```
User → WAF → CloudFront → S3 (website bucket)    [content]
                        → S3 (ops bucket)        [error pages + logs]
```

Security Layer — AWS WAF

AWS Web Application Firewall (WAF) is a security service that can filter the malicious requests and provide protection from intruder. It can block your application from attacks like SQL injection, cross site scripting and more. We can add rate limits to get protection from DDOS attacks. We can restrict access from countries, add captcha and other rules based on our use case.

- AWS WAF must be **us-east-1** for CloudFront
- We have added Rate Limiting rule to allow 20 requests in 5 minutes.

WAF Rules we have added:

- Common Rule Set: Blocks SQL injection, XSS, and other OWASP Top 10 attacks
- Known Bad Inputs: Blocks requests with malicious patterns
- Rate Limiting: Blocks IPs making more than 20 requests per 5 minutes
- WAF rules to block or allow access from specific countries
- We can allow or deny specific IP address or range.

Module "waf"

This module sets up a Web Application Firewall (WAF) for a CloudFront distribution on AWS.

1. Main Resource: aws_wafv2_web_acl

The WAF is scoped to CLOUDFRONT (meaning it protects a CloudFront distribution globally) and its default action is **allow** — traffic is allowed unless a rule blocks it.

2. Rules (in priority order)

Priority	Rule	Action	Purpose
1	AWSManagedRulesCommonRuleSet	Block (with body size override)	General protections (XSS, bad requests, etc.)
2	AWSManagedRulesKnownBadInputsRuleSet	Block	Blocks known exploit patterns (Log4j, etc.)
3	RateLimitRule	Block	Blocks IPs exceeding 2000 req / 5 min
4	BotControlRuleSet	Block (COMMON level)	Detects and blocks common bots
5	AmazonIpReputationList	Count only	Flags IPs with bad reputation (not blocking yet)
6	AnonymousIpList	Block	Blocks VPNs, Tor, proxies
7	SQLRuleSet	Block	SQL injection protection
8	LinuxRuleSet	Count only	Detects Linux-specific exploits (monitoring mode)
9	UnixRuleSet	Count only	Detects Unix-specific exploits (monitoring mode)
10	WindowsRuleSet	Block (with shell override)	Windows exploit protection
11	block-bad-user-agents	Block	Blocks scanners like sqlmap and nikto

> There's also a commented-out geo-restriction rule that would block traffic from specific countries if enabled.

A Few Things Worth Noting

- **Count vs. Block:** Several rules (IP reputation, Linux, Unix) are set to count instead of block. This is a common pattern when you want to observe traffic first before committing to blocking it.
- **Rule overrides:** Some rules have specific sub-rules overridden to allow (e.g., `SizeRestrictions_BODY`, `WindowsShellCommands_BODY`), likely to prevent false positives on legitimate large uploads or specific request patterns.

3. Supporting Resources

- **aws_wafv2_web_acl_logging_configuration** — Enables logging, but only keeps BLOCK and COUNT events (drops ALLOW logs to reduce volume).

- **aws_cloudwatch_log_group** — Creates a log group named `aws-waf-logs-<prefix>` (the `aws-waf-logs-` prefix is required by AWS) with a 7-day retention period.

CloudWatch Dashboard for CloudFront monitoring

Creating CloudWatch Dashboard and alarms. We have created alarms for 4xx and 5xx errors. Ideally, in production we usually monitor 403 and 5xx alerts since webapp domains.

Module "cloudwatch"

1. CloudWatch Dashboard

This module sets up AWS CloudWatch monitoring for a CloudFront distribution. Creates a visual dashboard with three panels:

- **CloudFront Performance** — tracks Cache Hit Rate, Origin Latency, and total Requests
- **Error Rates** — tracks 4xx (client errors) and 5xx (server errors) percentages
- **Data Transfer** — tracks bytes downloaded and uploaded

All panels use a 5-minute interval (`period = 300`) and display data as time-series graphs.

2. CloudWatch Alarms (4 total)

Each alarm watches a specific metric and fires if a threshold is breached over multiple evaluation periods. They all notify via an SNS topic (`var.sns_topic_arn`):

Alarm	Condition	Sensitivity
high_4xx_error_rate	4xx errors > 5%	2 periods
high_5xx_error_rate	5xx errors > 1%	2 periods
low_cache_hit_rate	Cache hit rate < 80%	3 periods
high_origin_latency	Origin latency > 3s	3 periods

The 5xx threshold is much tighter than 4xx (1% vs 5%) because server-side errors are generally more serious than client-side ones. The cache and latency alarms require 3 consecutive breaching periods before firing, making them less sensitive to short spikes.

One thing worth noting: CloudFront metrics must be read from `us-east-1` regardless of where your distribution serves traffic — that's hardcoded and correct, since CloudFront metrics are only published to that region.

Amazon SNS Topic for CloudFront Alerts

Module "sns"

This module sets up an AWS SNS (Simple Notification Service) notification system for CloudFront alerts. There are two resources:

1. aws_sns_topic — The Notification Channel

This creates an SNS Topic, which acts as a messaging hub — think of it as a broadcast channel. Anyone subscribed to it will receive messages published to it.

- name — dynamically built using a prefix variable (e.g., `myapp-cloudfront-alerts`)
- tags — merges common shared tags with topic-specific ones (Name, Type, Purpose) for resource organization/billing

2. aws_sns_topic_subscription — The Email Subscribers

This subscribes one or more email addresses to the topic above.

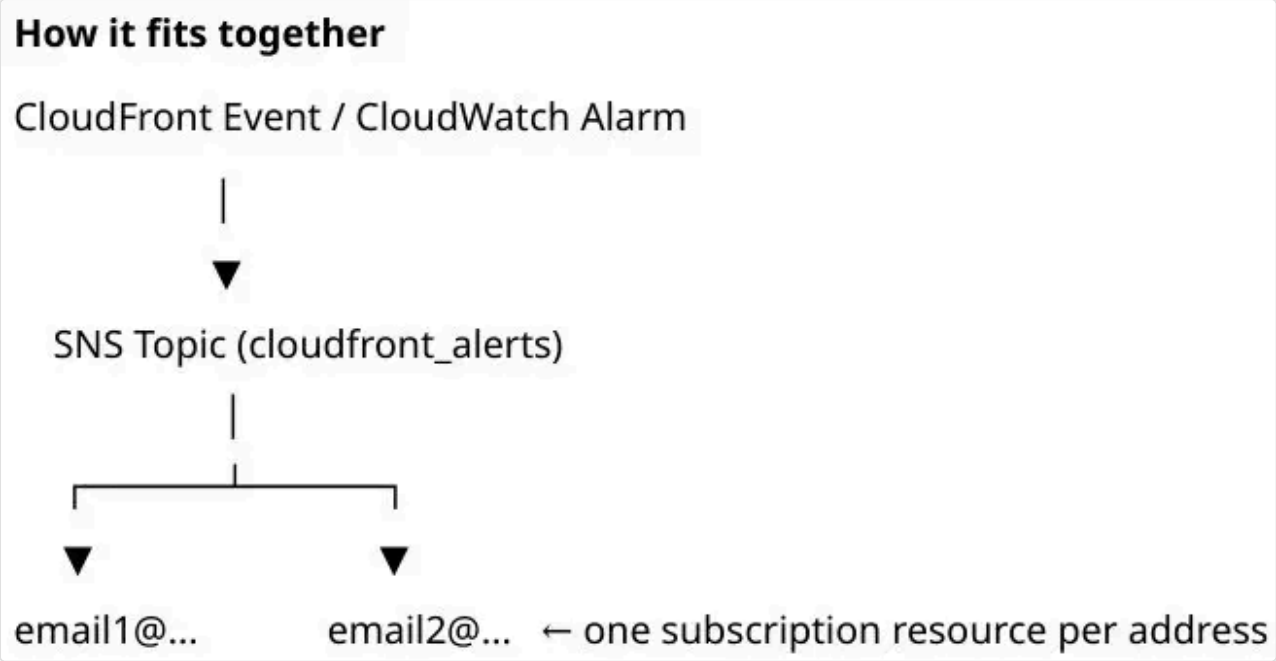


Figure 3 — CloudWatch alarm connected to SNS email notification

When something publishes an alert to this SNS topic (e.g., a CloudWatch alarm detecting high error rates on CloudFront), every subscribed email address gets notified.

> AWS will send a confirmation email to each address before they start receiving alerts — subscriptions aren't active until confirmed.

The outputs

After deployment, you'll get:

- Website URLs (https://cloudwithdarvin.com)
- Website URLs (https://www.cloudwithdarvin.com)

```
Apply complete! Resources: 151 added, 0 changed, 0 destroyed.
Outputs:
acm_certificate_arn             = "arn:aws:acm:us-east-1:583524027596:certificate/1ae4fd9b-817
f-486e-b18c-d7d6f3b27620"
backend_dynamodb_table_name     = "ce-dev-terraform-locks"
backend_s3_bucket_name          = "ce-dev-terraform-state-69127"
cloudfront_distribution_arn     = "arn:aws:cloudfront::583524027596:distribution/E309XEAH0SZK7
C"
cloudfront_distribution_id      = "E309XEAH0SZK7C"
cloudfront_domain_name          = "d2ks6bor1fo409.cloudfront.net"
cloudfront_hosted_zone_id       = "Z2FDTNDATAQYW2"
ds_record                       = "181711326793367F2950CE2ECC0A156655F7FD81..."
ds_record_KSK                   = "18171 ECDSA2P256SHA256 SHA-256 6793367F..."
ksk_status                      = "ACTIVE"
logs_bucket_name                = "ce-dev-cloudfront-ops-j49yrwm2"
route53_zone_id                 = "Z08158482U0KFY9CWWD9M"
waf_web_acl_arn                 = "arn:aws:wafv2:us-east-1:583524027596:global/webacl/..."
website_bucket_name             = "ce-dev-static-website-j49yrwm2"
website_url                     = "https://cloudwithdarvin.com"
www_website_url                 = "https://www.cloudwithdarvin.com"
```

Step 4: Deploy content and invalidate cache

The final stage separates content deployment from infrastructure, allowing static website updates without re-running Terraform. Static HTML files are in the content folder, and a GitHub Actions workflow (`deploy-infrastructure.yml`) automates deployment on every merge to the main branch.

Workflow GitHub Actions

- **terraform-checkov.yml** — Automated security scanning framework using Checkov to detect Terraform misconfigurations at both repository and pull request (PR) levels.
- **deploy-infrastructure.yml** — Terraform workflow to provision infrastructure and deploy Static Website with OIDC, synchronize Website content if something changed and invalidates a CloudFront cache, forcing edge servers to fetch fresh content from the origin.

Invalidate CloudFront cache

Create a CloudFront cache invalidation for all paths (`/*`). Without this step, CloudFront would continue serving the previously cached version for up to 24 hours (the default TTL of the Managed-CachingOptimized cache policy). The invalidation ensures visitors see the updated content immediately.

What is Cache Invalidation?

Cache invalidation removes files from CloudFront edge caches before they expire naturally.

When to Use Invalidation:

- Emergency fixes: Critical bug fixes that can't wait for cache expiry
- Content updates: When you need immediate content changes
- Security patches: Urgent security updates
- Marketing campaigns: Time-sensitive content launches

We should always provide specific path or files to invalidate.

What happens under the hood of

1. CloudFront marks all cached objects as stale across its edge locations
2. The next request to each edge node triggers a fresh fetch from your origin (S3, EC2, etc.)
3. Subsequent requests are served from the refreshed cache
4. This pattern is common in **static site deployments** — after uploading new files to S3, you invalidate the CloudFront cache so users immediately get the updated content instead of stale cached versions.

Monitoring

The project includes:

CloudWatch Dashboard: Performance metrics and error rates

- CloudWatch Alarms:

    - High 4xx error rate (>5%)
    - High 5xx error rate (>1%)
    - Low cache hit rate (<80%)
    - High origin latency (>3 seconds)
- **Access Logs:** Detailed request logs stored in S3

Quick Start

Step 1 — Fork and clone

```
$ git clone https://github.com/darvin-rakotomalala/s3-static-website
$ cd s3-static-website
```

Step 2 — Configure Terraform

Edit *s3-static-website/dev-terraform.tfvars* to match your target region and preferences. No secrets go here — just region, domain name, and default tags.

Step 3 — Deploy infrastructure

```
$ cd s3-static-website
$ terraform init
$ terraform fmt -recursive
$ terraform validate
$ terraform plan -var-file="dev-terraform.tfvars" -no-color -out=TFplan.JSON
$ terraform apply -var-file="dev-terraform.tfvars" -auto-approve
```

Verification

- After deploying all three steps, the site was live at *https://cloudwithdarvin.com/* with a secure connection confirmed by the browser.

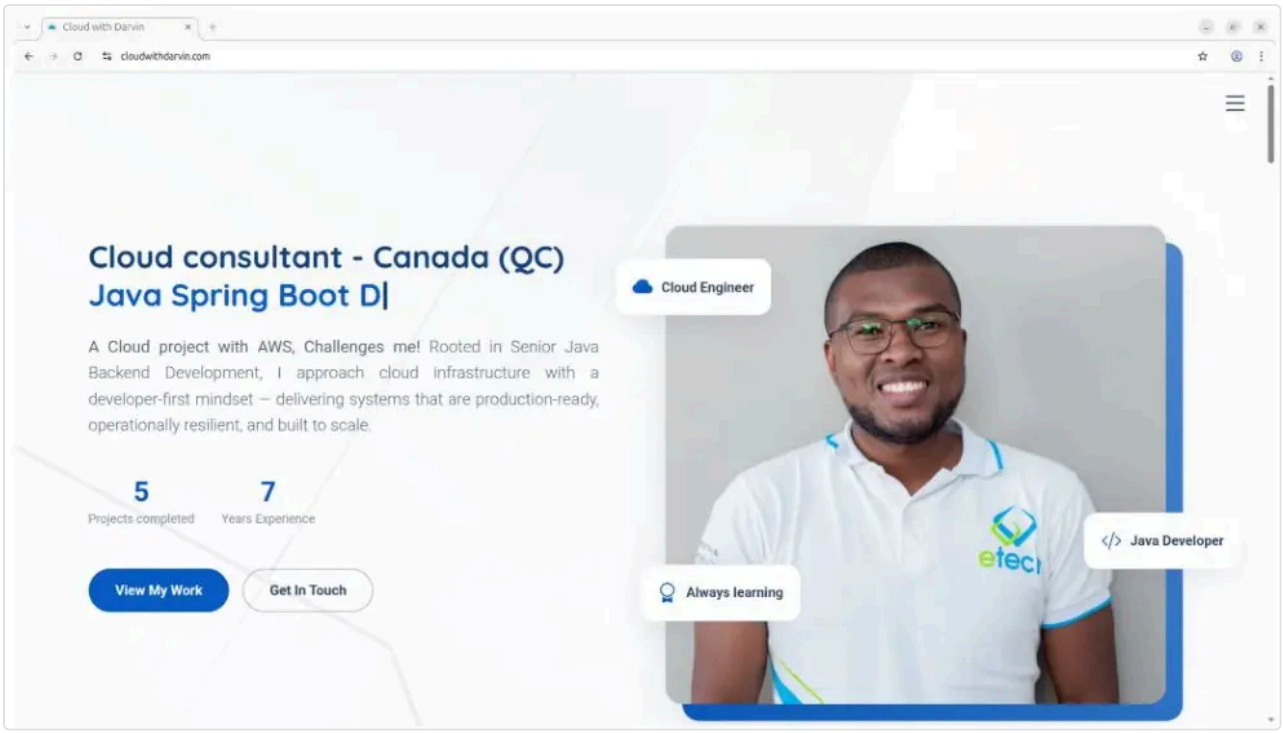


Figure 4 — Website live at https://cloudwithdarvin.com with valid TLS certificate

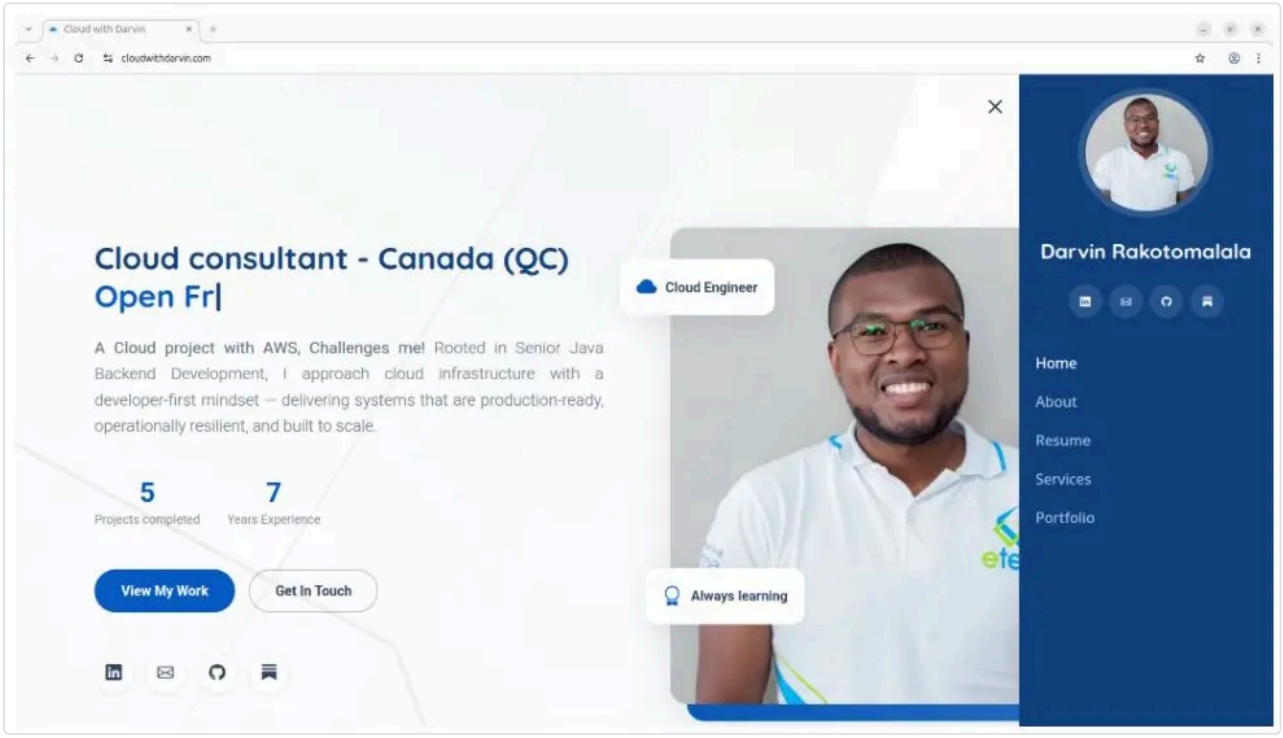


Figure 5 — Browser certificate inspector confirming ACM-issued TLS certificate

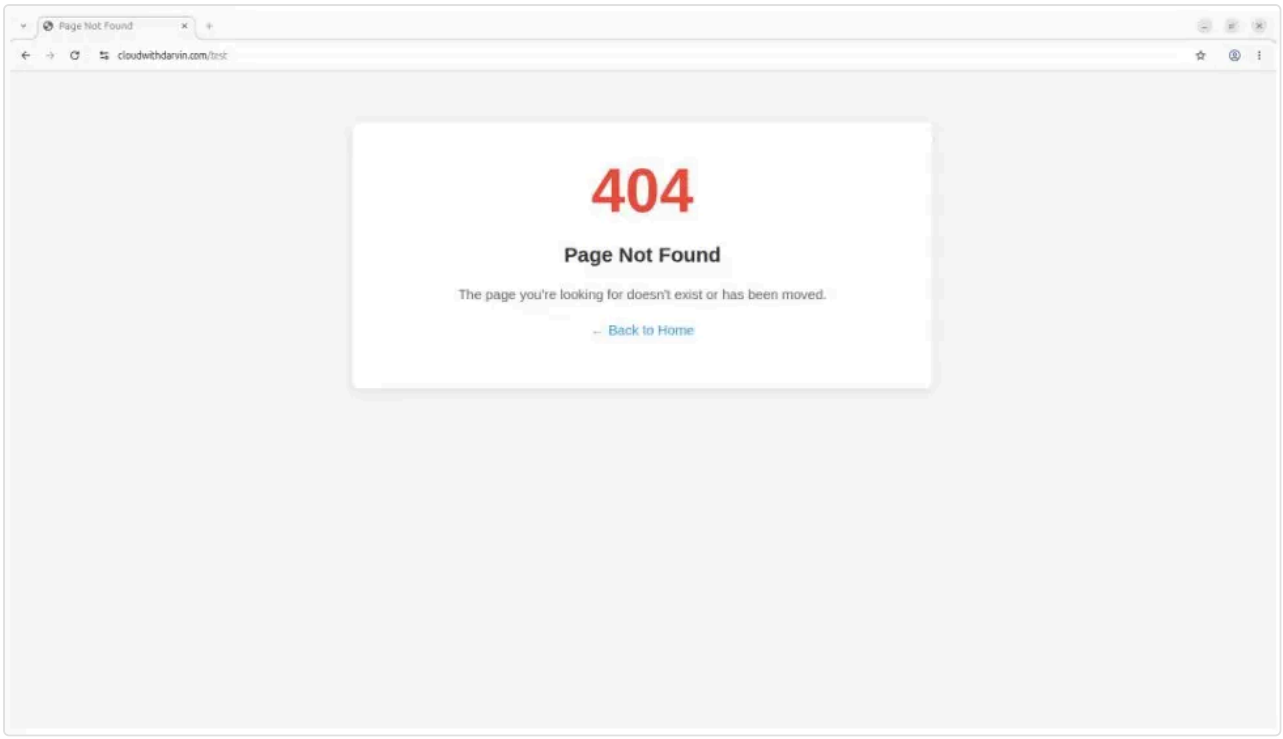


Figure 6 — Secure connection details confirmed

On the AWS console, the CloudFront distribution showed the alternate domain names (cloudwithdarvin.com and www.cloudwithdarvin.com), the attached ACM certificate, the TLSv1.2_2021 security policy, and standard logging enabled.

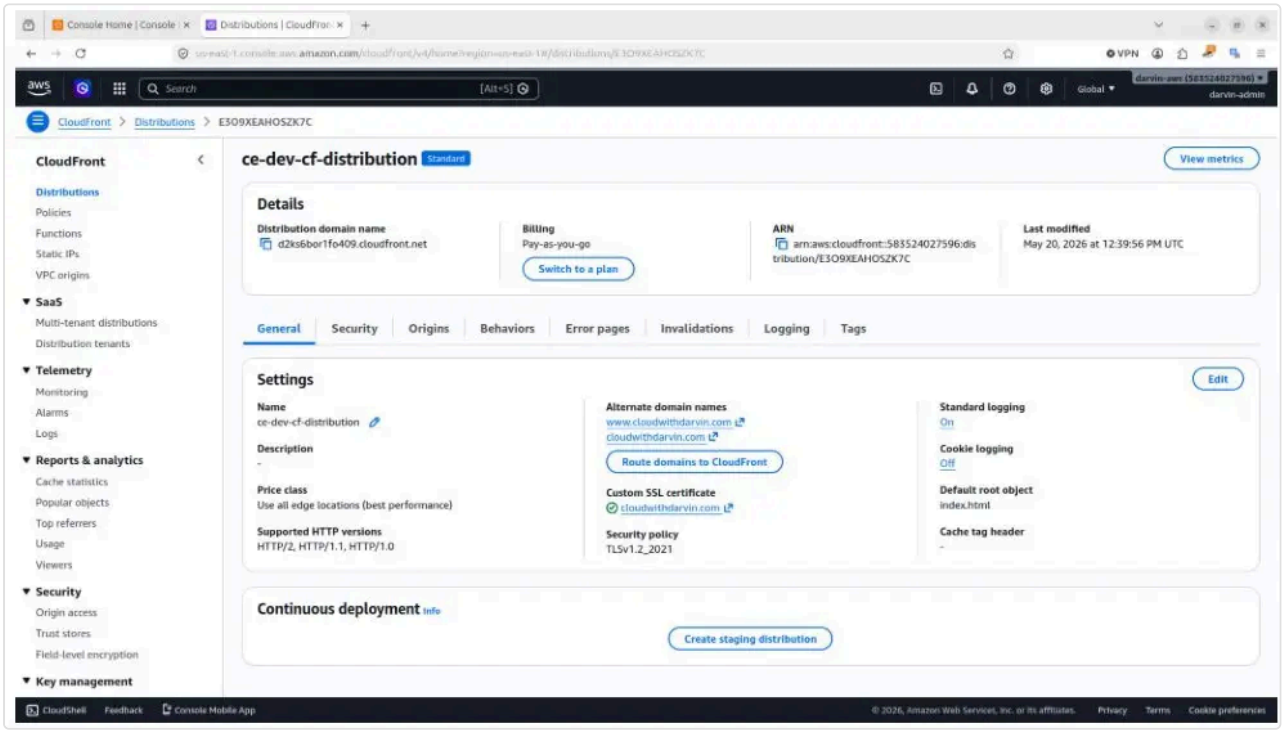


Figure 7 — CloudFront distribution with custom domain aliases and ACM certificate attached

The two S3 origins — one for website content and one for the ops bucket — were configured correctly.

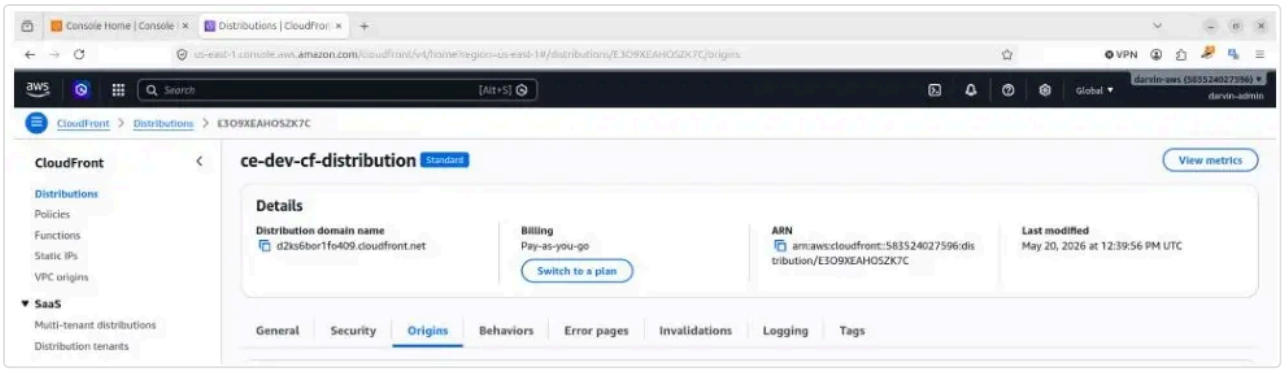


Figure 8 — Two S3 origins configured in CloudFront with OAC enabled

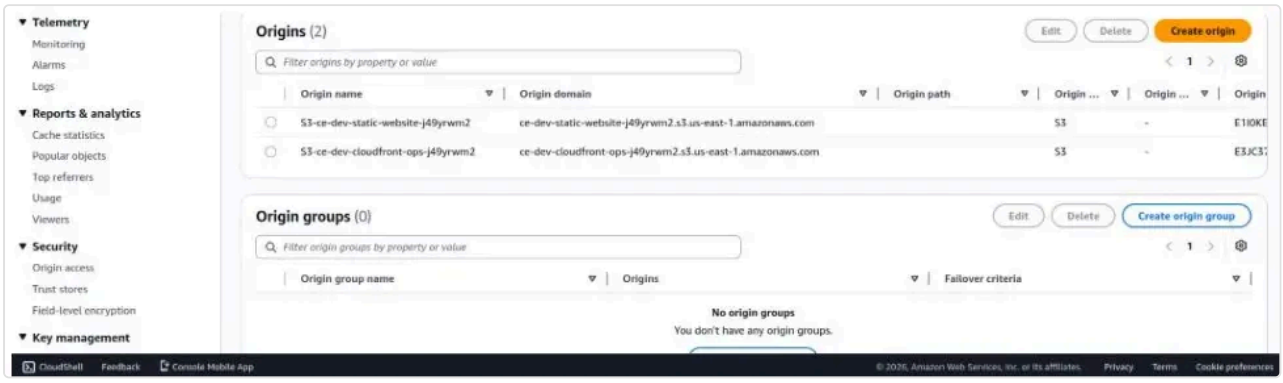


Figure 9 — CloudFront distribution details view

The custom error responses are in place — HTTP 403 and 404 errors serve the custom 404 page, while 500 errors serve the 500 page, both from the ops bucket's `/error-pages/` prefix.

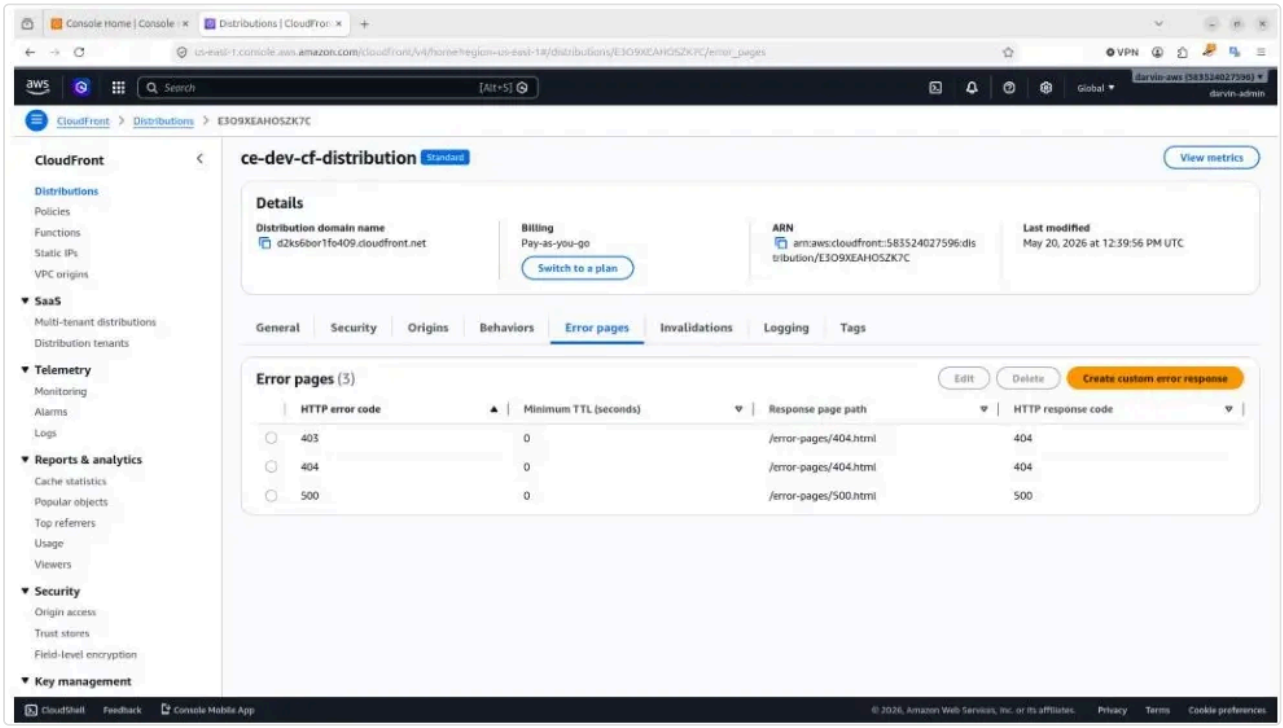


Figure 10 — Custom error response configuration in CloudFront

The Route 53 hosted zone shows the A records (root and www) aliased to CloudFront, the NS and SOA records, and the CNAME records created during ACM DNS validation.

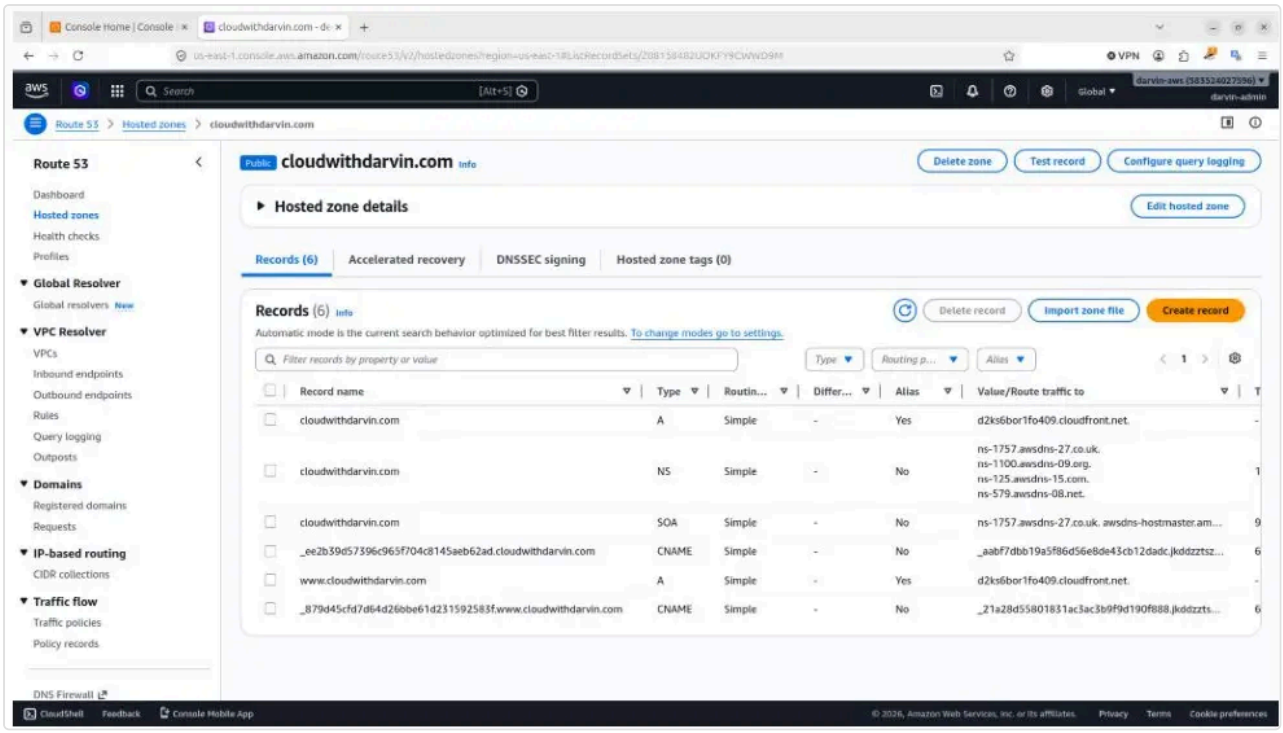


Figure 11 — Route 53 hosted zone showing A alias records and ACM CNAME validation records

DNSSEC signing

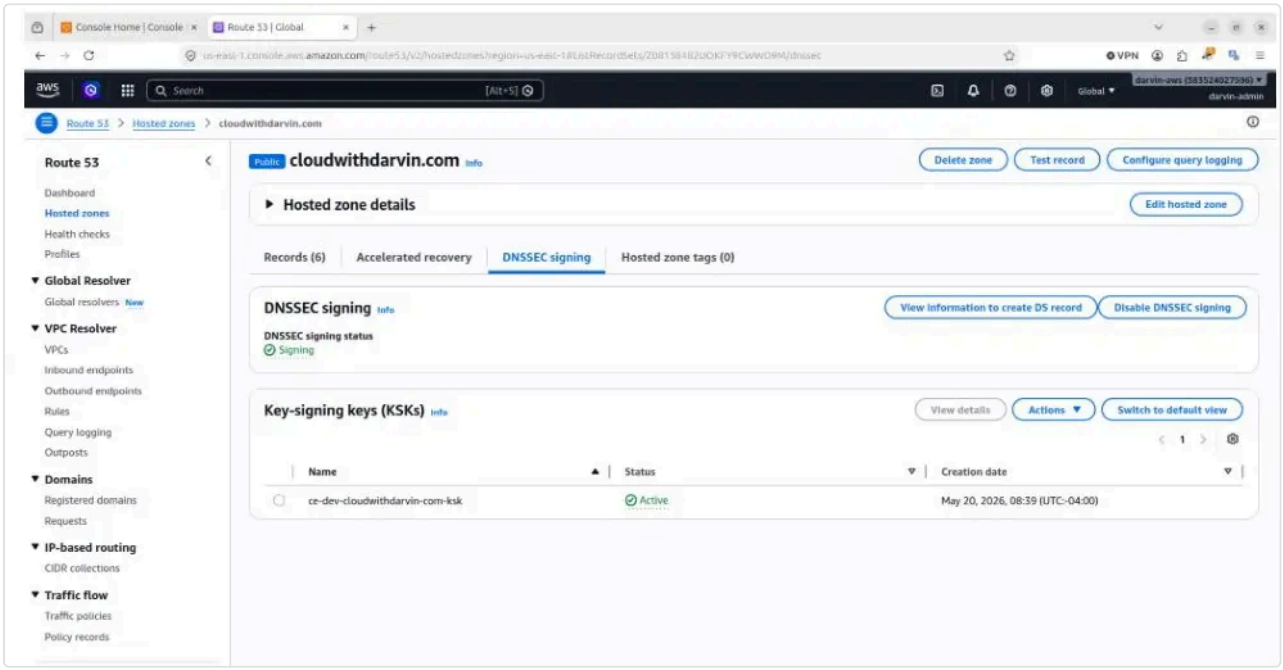


Figure 12 — DNSSEC signing active — Key Signing Key status ACTIVE in Route 53

WAF test

Manual HTTP Requests with curl — the quickest way to test specific WAF rules:

```
# Test a SQL injection rule
$ curl -i "https://cloudwithdarvin.com/?id=1' OR '1'='1"

# Test an XSS rule
$ curl -i "https://cloudwithdarvin.com/?q=<script>alert('xss')</script>"

# Test a path traversal rule
$ curl -i "https://cloudwithdarvin.com/../../../../etc/passwd"

# Test a blocked User-Agent (bad bot rule)
$ curl -i -A "sqlmap/1.0" https://cloudwithdarvin.com/

# Test rate limiting (send many requests quickly)
$ for i in {1..200}; do curl -s -o /dev/null -w "%{http_code}" https://cloudwithdarvin.com/; d
one
```

A **403** response means WAF blocked the request. A **200** means it passed through.

Automated Scanning with nikto or nuclei — run a full scan against your domain to see what WAF catches:

```
# Install nikto (web vulnerability scanner)
$ nikto -h https://cloudwithdarvin.com

# Or use nuclei for targeted WAF bypass testing
$ nuclei -u https://cloudwithdarvin.com -t waf-bypass/
```

Troubleshooting

- **The Problem:** css, js, img file are not loaded when accessed on my domain.
- **Root cause:** The HTML source files don't have metadata when uploading the assets files in website bucket.

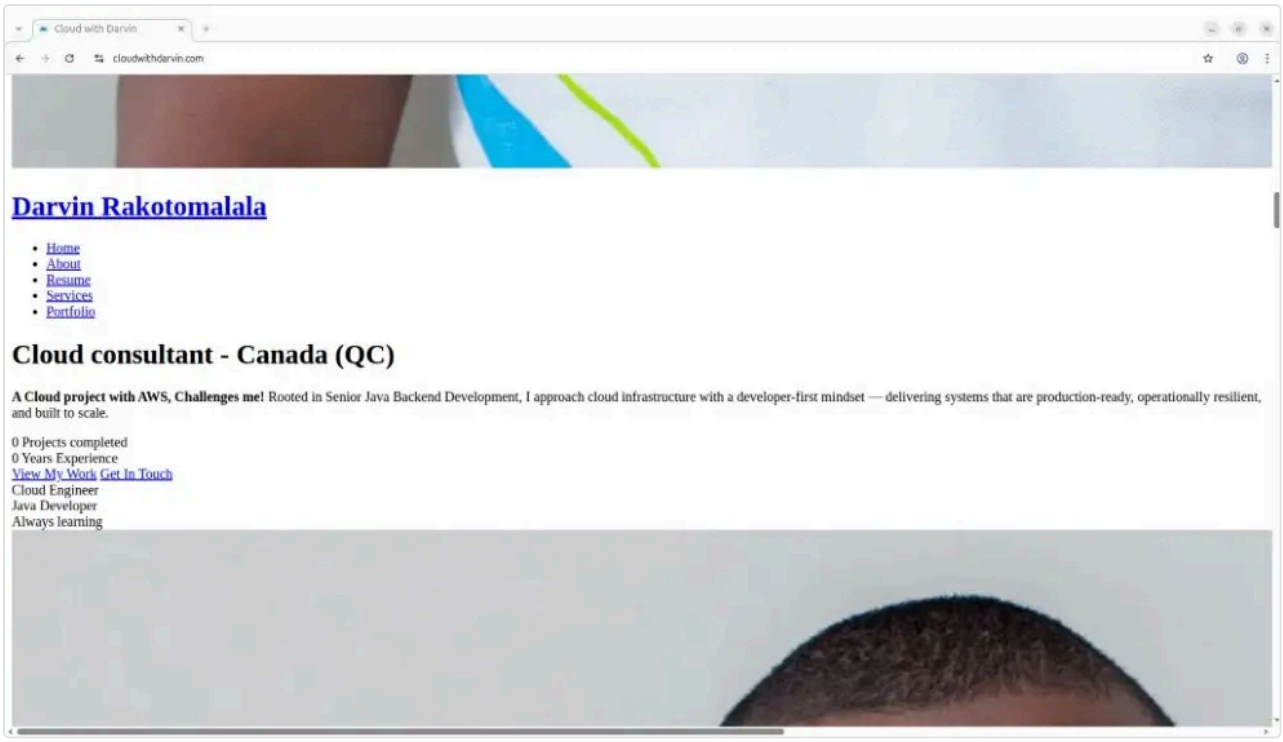


Figure 13 — Live portfolio website after resolving MIME type issue

Solution: In Terraform code sets a mime_types of each file.

```
locals {
  mime_types = {
    "html"  = "text/html"
    "css"   = "text/css"
    "js"    = "application/javascript"
    "json"  = "application/json"
    "webp"  = "image/webp"
    "png"   = "image/png"
    "jpg"   = "image/jpeg"
    "jpeg"  = "image/jpeg"
    "svg"   = "image/svg+xml"
    "ico"   = "image/x-icon"
    "woff"  = "font/woff"
    "woff2" = "font/woff2"
    "ttf"   = "font/ttf"
    "txt"   = "text/plain"
  }
}

resource "aws_s3_object" "provision_source_files" {
  bucket       = module.s3.s3_bucket_website_id
  for_each     = fileset("${var.source_files}/", "**/*.*")
  key          = each.value
  source       = "${var.source_files}/${each.value}"
  content_type = lookup(local.mime_types,
                   split(".", each.value)[length(split(".", each.value)) - 1],
                   "application/octet-stream")
  etag         = filemd5("${var.source_files}/${each.value}")
}
```

Cache Invalidation

After setting up the mime_types, CloudFront may still be serving cached content. Go to:

1. Invalidations tab → Create invalidation
2. Add path: `/*`
3. Submit and wait ~1 min

After invalidating on path (`/*`), the result becomes error page "Page Not Found"

Conclusion

In this project, we walked through deploying a production-grade secure static website on AWS using Terraform — provisioning DNS with Route 53, building the infrastructure with S3, CloudFront, ACM, KMS, WAF, and automating content deployment through GitHub Actions.

If you have a static site idea — a portfolio, documentation, a landing page — and were wondering how to host it on AWS with proper security and automation, this project has the blueprint.